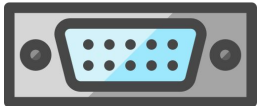


Comunicación serie

Clase 3
Fecha: 05/04/25026



Docentes:

- Dr. Lic. Federico Farfán - arfan.roberto.f@gmail.com
- Luis Mariano Campos - lmcampos@fi.uba.ar

Objetivos de la clase

- Comprender por qué las estructuras en C son útiles en sistemas embebidos
- Relacionar el uso de struct con la configuración de periféricos mediante HAL
- Comprender qué significa comunicación serial en sistemas embebidos
- Identificar las señales de comunicación UART: TX, RX y GND
- Interpretar los parámetros principales de una configuración UART:
- Reconocer la diferencia entre protocolo UART y niveles eléctricos de la señal
- Distinguir, a nivel introductorio, UART, SPI e I2C como interfaces de comunicación
- Comprender de manera conceptual la diferencia entre polling e interrupciones

¿Por qué hablar de estructuras antes de UART?

- STM32 HAL utiliza estructuras para agrupar parámetros de configuración de los periféricos.
- Ejemplos de parámetros que suelen agruparse:
 - Pin utilizado
 - Modo de operación
 - Velocidad
 - Resistencias internas
 - Baud rate
 - Paridad
 - Bits de stop
- Comprender struct permite leer y modificar con mayor seguridad el código generado por STM32CubeIDE.

¿Qué es una estructura en C?

```
typedef struct {  
    char nombre[30];  
    int dni;  
    int edad;  
    float altura;  
} Persona_t;  
  
Persona_t persona1;  
Persona_t persona2;  
  
strcpy(persona1.nombre, "Ana");  
persona1.dni = 30123456;  
persona1.edad = 25;  
persona1.altura = 1.75;  
  
strcpy(persona2.nombre, "Luis");  
persona2.dni = 32876543;  
persona2.edad = 30;  
persona2.altura = 1.68;
```

Ejemplo más cercano a sistemas embebidos

Configuración de GPIO

```
typedef struct {  
  
    int pin;  
    int modo;  
    int pull;  
    int velocidad;  
  
}GPIO_Simulado_t;
```

```
GPIO_Simulado_t gpio_led_rojo;  
  
gpio_led_rojo.pin = 5;  
gpio_led_rojo.modo = 1;  
gpio_led_rojo.pull = 0;  
gpio_led_rojo.velocidad = 1;
```

Ejemplos de estructura código fuente

Relación con STM32 HAL

STM32 HAL también utiliza estructuras.

La estructura agrupa los parámetros de configuración del periférico.

```
GPIO_InitTypeDef GPIO_InitStructure = {0};
```

```
GPIO_InitStructure.Pin = GPIO_PIN_5;
```

```
GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
```

```
GPIO_InitStructure.Pull = GPIO_NOPULL;
```

```
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);
```

Introducción general a la comunicación serial

¿Por qué un sistema embebido necesita comunicarse?

- microcontrolador ↔ PC
- microcontrolador ↔ sensor
- microcontrolador ↔ memoria
- microcontrolador ↔ otro microcontrolador
- microcontrolador ↔ módulo externo

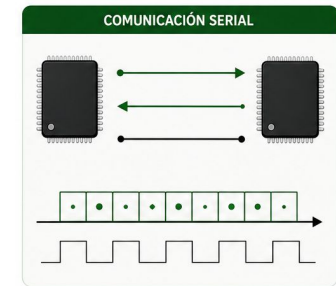
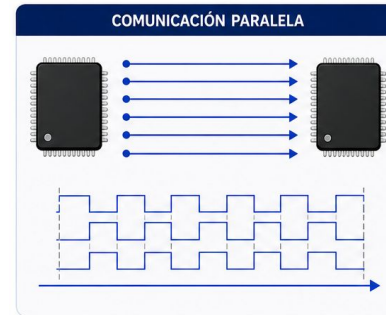


Comunicación paralela vs. serial

- Paralela: varios bits al mismo tiempo
- Serial: un bit detrás de otro

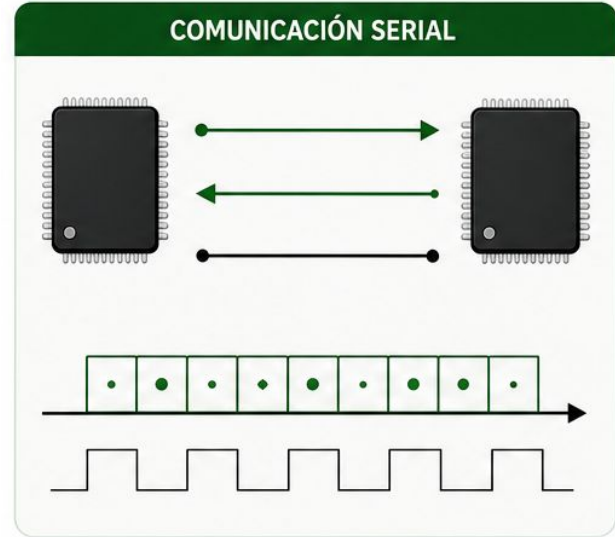
¿Por qué la comunicación serial es tan usada?

- menos cables
- menor costo
- mayor simplicidad física
- mejor adaptación a placas y dispositivos compactos



UART: Universal Asynchronous Receiver Transmitter

- Comunicación serial **asíncrona**
- No utiliza señal de reloj compartida
- Transmisión y recepción mediante:
 - **TX**
 - **RX**
- Ambas partes deben acordar:
 - baud rate
 - cantidad de bits
 - paridad
 - bits de stop

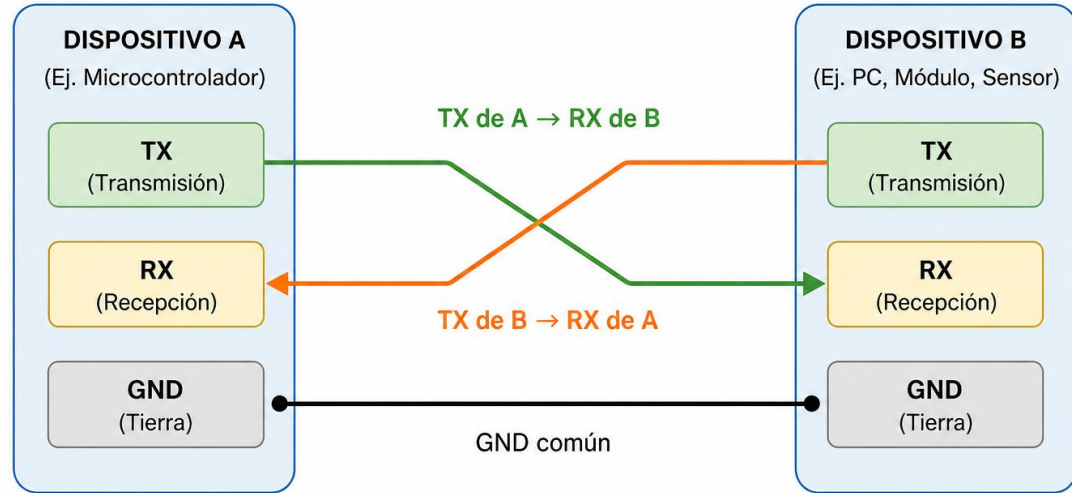


Señales básicas de UART

Señales principales

- **TX:** línea de transmisión
- **RX:** línea de recepción
- **GND:** referencia común

UART – Conexión básica



- TX de A conectado a RX de B
- TX de B conectado a RX de A
- GND común entre ambos dispositivos

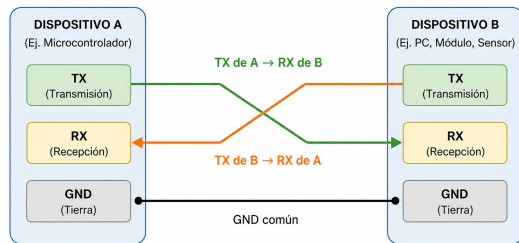


Para que UART funcione correctamente, ambos dispositivos deben compartir la misma referencia de GND.

Trama UART

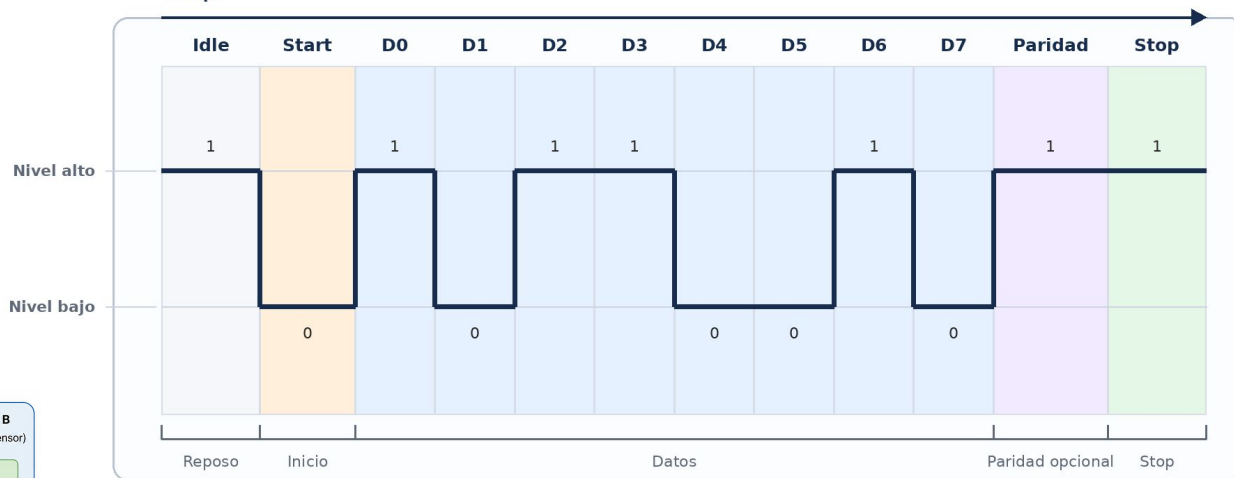
Estructura típica de una trama UART

- 1 bit de inicio
- 5 a 9 bits de datos
- paridad opcional
- 1 o más bits de stop



UART - Trama típica

Ejemplo conceptual: reposo, bit de inicio, 8 bits de datos, paridad opcional y bit de stop



La línea UART permanece en reposo en nivel alto. La transmisión comienza con un bit de inicio en bajo.

Parámetros que deben coincidir

Para que UART funcione correctamente::

- Baud rate
- Cantidad de bits de datos
- Paridad
- Bits de stop

Ejemplo frecuente

- 9600 baudios
 - 8 bits de datos
 - Sin paridad
 - 1 bit de stop
- 9600 8N1



Niveles eléctricos

Comparación entre UART TTL/CMOS y RS-232

UART TTL / CMOS

Niveles típicos de microcontrolador

Estado	Nivel	Tensión
1 lógico	Nivel alto	3,3 V / 5 V
0 lógico	Nivel bajo	0 V

La tensión exacta depende del dispositivo.

RS-232

Niveles eléctricos del estándar

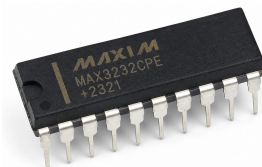
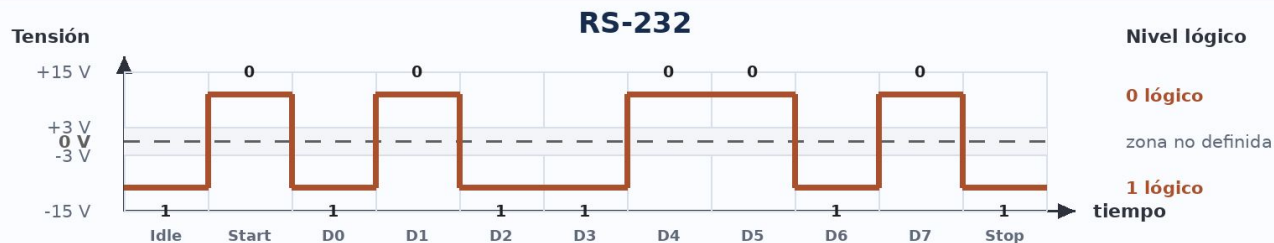
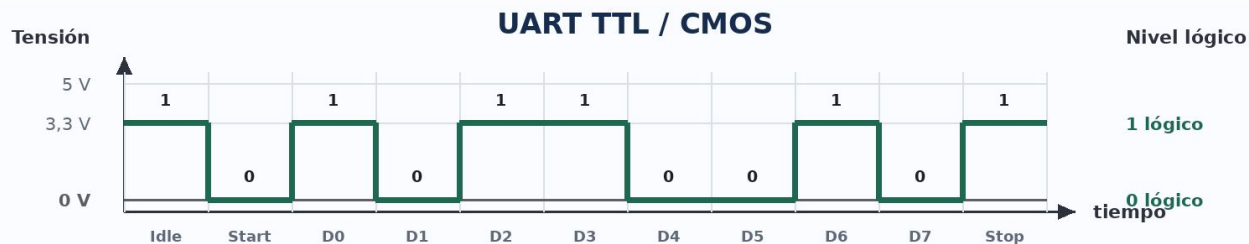
Estado	Nivel eléctrico	Tensión
0 lógico	Positivo	+3 V a +15 V
Indefinido	Zona no válida	-3 V a +3 V
1 lógico	Negativo	-3 V a -15 V

No se conecta directamente a una UART de microcontrolador.

Mismo intercambio serie, distintos niveles eléctricos.

Niveles eléctricos

UART TTL/CMOS y RS-232: misma secuencia lógica, distintos niveles de tensión



En RS-232, los estados válidos se ubican por encima de +3 V y por debajo de -3 V.
Por eso no se conecta directamente a una UART TTL/CMOS de microcontrolador.

Ventajas, limitaciones y usos de UART

Ventajas

- Simple de implementar
- Pocas líneas
- Muy útil para depuración
- ampliamente disponible en microcontroladores

Limitaciones

- Típicamente punto a punto
- No tiene reloj compartido
- Depende de una configuración coincidente

Usos comunes

- Comunicación con PC
- Terminal serie
- Módulos Bluetooth
- GPS
- Debug

¿Cuándo elegir UART?

Elegir UART cuando:

- Se necesita una comunicación simple
- Hay pocos dispositivos involucrados
- Se desea enviar mensajes de texto o debug
- Se trabaja con terminales serie o módulos compatibles

➔ UART es ideal cuando se prioriza simplicidad.

Código fuente: STM32CubeIDE

Más allá de UART: SPI e I2C

Existen diferentes formas de comunicación serie. Cada protocolo tiene características y aplicaciones típicas.

SPI

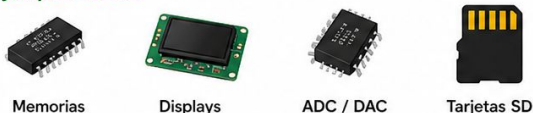
Comunicación serie síncrona

- Alta velocidad
- Un controlador y uno o varios periféricos
- Transmisión y recepción simultánea
- Requiere una línea de selección por periférico (CS / NSS)

Señales típicas



Ejemplos de uso

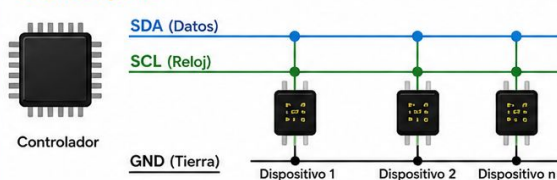


I2C

Comunicación serie síncrona

- Bus compartido con direccionamiento
- Permite varios dispositivos con pocas líneas
- El controlador inicia la comunicación
- Ideal para conectar muchos dispositivos de baja / media velocidad

Señales típicas



Ejemplos de uso

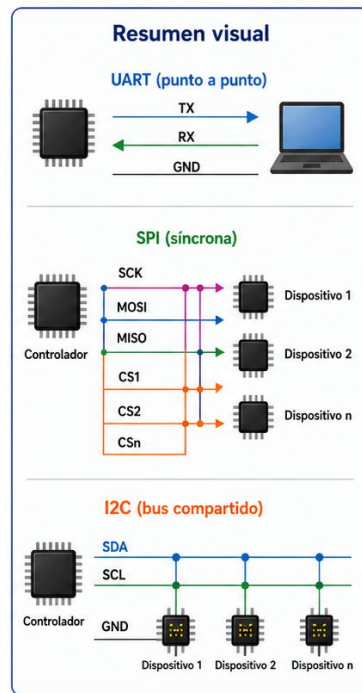


En esta materia veremos SPI e I2C en forma detallada más adelante.
Por ahora, los presentamos a nivel general para reconocer sus características y aplicaciones típicas.

Comparación general entre UART, SPI e I2C

Cada protocolo tiene características propias y se utiliza en situaciones diferentes.

Característica	UART	SPI	I2C
 Tipo de comunicación	Asíncrona	Síncrona	Síncrona
 Reloj compartido	No	Sí (SCK)	Sí (SCL)
 Líneas básicas	TX, RX, GND	SCK, MOSI, MISO, CS, GND	SDA, SCL, GND
 Conexión típica	Punto a punto	Controlador - Periféricos (cada uno con su CS)	Bus compartido (varios dispositivos)
 Velocidad típica	Media	Alta	Media / Baja
 Uso típico	PC, debug, módulos Bluetooth, GPS, terminal serie	Memorias, displays, ADC/DAC, tarjetas SD	Sensores, RTC, EEPROM, expansores de E/S
 Ventajas principales	Simple, pocas líneas, muy extendida	Muy rápida, full dúplex	Pocas líneas, permite muchos dispositivos
 Limitaciones principales	Típicamente punto a punto, velocidad limitada	Requiere más líneas (incluye CS), no ideal para muchos dispositivos	Velocidad limitada, más sensible al ruido, cable más corto



No existe una única "mejor" opción: la elección depende del periférico a usar, la velocidad requerida, la cantidad de dispositivos y la cantidad de pines disponibles.

¿Cómo se entera el microcontrolador de que ocurrió un evento?



En sistemas embebidos, los periféricos pueden generar eventos.
¿Cómo detecta el programa que ese evento ocurrió?

Situaciones típicas



Llegó un dato
por UART



Terminó una
conversión del ADC



Venció un
temporizador



Cambió el estado
de un pin



Finalizó una
transferencia SPI o I2C



Pregunta clave

¿Cómo se entera el programa de que ese evento ocurrió?

Dos estrategias principales



Polling

El programa pregunta
repetidamente



Interrupciones

El periférico avisa al
procesador cuando ocurre el evento



A lo largo de la clase vimos los periféricos y cómo se configuran.
Ahora veremos cómo el microcontrolador puede detectar sus eventos.

Polling vs interrupciones

Dos formas de manejar eventos generados por periféricos.

Aspecto	 POLLING	 INTERRUPCIONES
 Idea principal	El programa pregunta repetidamente si ocurrió un evento.	El periférico avisa al procesador cuando ocurre un evento.
 Ventaja	Simple de entender e implementar.	Más eficiente y reactivo. El CPU puede hacer otras tareas hasta que ocurre el evento.
 Limitación	Puede desperdiciar tiempo de CPU (pregunta continuamente aunque no haya nada que atender).	Requiere más cuidado en el diseño: uso de ISR, variables compartidas, secciones críticas, etc.
 Uso típico	Sistemas simples o cuando el evento no es crítico en tiempo.	Sistemas complejos o con múltiples tareas, eventos frecuentes o críticos en tiempo.
 Ejemplo con UART	El programa revisa constantemente si llegó un dato (RXNE = 1). Si llegó, lo lee y continúa.	La UART genera una interrupción cuando llega un dato. La rutina de interrupción (ISR) lo lee y lo procesa.



Idea clave



Polling: el procesador pregunta.



Interrupción: el periférico avisa.



Polling: mirar el celular cada pocos segundos para ver si llegó un mensaje.



Interrupción: esperar la notificación. El sistema nos avisa cuando ocurre el evento.

Analogía



La elección depende del sistema, de la frecuencia de los eventos y de los requisitos de tiempo y rendimiento.

Enunciado: opcional

Se desea simular la configuración de una comunicación UART entre un microcontrolador y un módulo externo.

Para que la comunicación funcione correctamente, ambos dispositivos deben coincidir en:

- baud rate;
- cantidad de bits de datos;
- paridad;
- bits de stop.

Se pide definir una estructura `UART_Config_t` que permita representar esos parámetros.

Luego, crear dos configuraciones:

- `uart_micro;`
- `uart_modulo.`

Finalmente, implementar una función que compare ambas configuraciones e indique si son compatibles o no.